Discover Data

RESEARCH Open Access

Debugmate: an AI agent for efficient on-call debugging in complex production systems

Rohan Modi[1*], Navtej Reddy[2] and Sai Sreenivas Kodur[3]

*Correspondence:
Rohan Modi
rohanmodi@gatech.edu
[1]Georgia Institute of Technology, Atlanta 30332, GA, United States
[2]Observe.AI, Bangalore 560103, Karnataka, India
[3]Observe.AI, Redwood City 94065, California, United States

Abstract
Production systems are often complex and distributed in nature. Debugging such complex systems is often time-consuming and nuanced, requiring software engineers (SEs) to understand vast codebases and the context of their organization's engineering setup. This leads to a huge cognitive load on SEs, particularly while they are on-call where they are expected to fix critical time-sensitive problems. This often causes sub-optimal human performance, which in turn decreases productivity and potentially impacts system reliability due to prolonged downtimes. In this paper, we introduce DebugMate (Observe.AI. Debugmate. 2024. https://github.com/Observeai-Research/DebugMate), an AI Agent that uniquely integrates an organization's internal context with external knowledge sources. DebugMate connects to the organization's key system resources like documentation, codebase, and knowledge base of historical incidents, along with online developer platforms (eg. StackOverflow, GitHub) and helps SEs respond faster by generating multiple hypotheses for identifying the root cause of a production issue. DebugMate employs Retrieval-Augmented Generation (RAG), ReAct, Tree-of-Thought, and long-term memory to provide grounded hypotheses for debugging via structured and systematic self-planning. In addition, it utilizes graphical representations to build context of not only the organization's code but also of imported code modules, to specifically identify and resolve complex issues that may be caused by unfamiliar external frameworks (eg, SpringBoot). Our proposed approach increases accuracy in identifying an issue by 20% compared to the baseline. On our historical reliability incidents, DebugMate achieves a 77% success rate in identifying root causes and suggesting fixes.

Keywords AI Debugging Agent, Complex production systems, On-call debugging, Software reliability, Code Dependency Management, Software engineering

1 Introduction

Existing automated debugging methods have made significant strides in resolving logical code issues, but they often fall short in addressing the complexities of modern production environments [1]. These environments, characterized by dynamic components and decentralized development practices, require a more comprehensive approach to debugging that goes beyond traditional code analysis [2]. While tools like SWE-Agent [3] excel at identifying logical errors within codebases, they may struggle with niche bugs

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

that demand a deeper understanding of organization-specific contexts–information on which large language models (LLMs) are not typically pre-trained.

Other existing tools [4, 5] either attempt to process entire codebases as context or rely on user-selected code segments, which can limit their efficiency and ability to autonomously navigate through multiple codebases. These approaches fail to address the full scope of challenges faced by software engineers (SEs) during on-call situations, where quick resolution of critical production issues is paramount [6]. To address these limitations, we introduce DebugMate, an AI Debugging Agent that uniquely integrates an organization's internal context with external knowledge sources. DebugMate connects to key system resources such as documentation, codebases, and historical incident knowledge bases, as well as online developer platforms like StackOverflow [7] and GitHub [8]. This comprehensive approach allows DebugMate to provide SEs with a fuller context of their organization's engineering setup, significantly reducing the cognitive load associated with debugging complex, distributed systems.

DebugMate employs a combination of advanced techniques, including Retrieval-Augmented Generation (RAG), ReAct [9], Tree-of-Thought reasoning [10], and long-term memory to generate multiple grounded hypotheses for identifying the root cause of production issues. The agent utilizes structured and systematic self-planning to navigate through complex production systems, querying relevant tools to provide the required system context. DebugMate's key innovations include:

- Utilizing graphical representations to build context not only of the organization's code but also of imported code modules [11]. This feature is particularly valuable for identifying and resolving complex issues that may be caused by unfamiliar external frameworks, such as SpringBoot.
- Gathering context about the issue by querying tools that provide information from key organizational sources. This includes knowledge of historical incidents, recently merged code changes, and issue details from developer portals.
- Employing Tree-of-Thoughts to explore multiple reasoning paths and hypothesize potential problem areas across different system components. This keeps the agent focused on individual branches while allowing it to handle multiple parallel code implementation paths.
- Leveraging both short-term memory to retain important insights from previous thoughts and long-term memory to store valuable information like code dependencies and class hierarchies [12]. This helps DebugMate offer a more holistic and efficient approach to automated debugging in complex production environments.

In the subsequent section, we present DebugMate's approach to debugging production systems, highlighting its context-gathering tools and Tree-of-Thought architecture for systematic self-planning in the debugging process.

2 Methodology

To address the challenges of debugging complex production environments, we developed DebugMate, an AI Agent that mimics and enhances the problem-solving approach of experienced software engineers (SEs). Our methodology is rooted in the understanding that effective debugging in modern, distributed systems requires a holistic approach

that combines internal organizational knowledge, external expertise, and advanced AI techniques. The debugging process implemented by DebugMate follows a structured yet flexible approach:

- *Context Gathering* DebugMate begins by collecting comprehensive context about the issue, including stack traces, system state, and historical incidents. This step leverages both internal resources and external developer platforms to build a rich understanding of the problem space [13].
- *Hypothesis Generation and Solution Formation* Using the gathered context, DebugMate generates multiple hypotheses regarding the potential causes of the bug. This process is enhanced by the integration of Retrieval-Augmented Generation (RAG), ReAct, and Tree-of-Thought reasoning techniques. Next, DebugMate formulates one or more potential solutions, prioritizing them based on their likelihood of resolving the issue.

2.1 Context gathering

In this section, we outline the library of tools we built out to gather organization context.

- *Code Search* Takes a string representing a class name and construct name as input to find exact matches. It utilizes a vertical Path Tool to map class names from the stack trace to file locations and uses an Abstract Syntax Tree (AST) to map field objects to their respective class types. This allows querying object names to locate the necessary constructs. For example, if the agent queries *entityService.teamEntityResponse*, the Code Search tool first identifies the object *entityService* as type *BulkUserService* and then uses the Path tool to locate the corresponding class file. Additionally, an AST identifies *BulkUserService* as an interface, so an interface-implementation mapping is used to find the queried *teamEntityResponse* function within the implemented class, *BulkUserServiceImpl*. DebugMate integrates a code search module designed to extract only the most relevant code segments from large Java files. When a function or class is too large for direct retrieval, the agent receives only the focused function body. If the content still exceeds the token limit, DebugMate dynamically prompts the user to paste a reduced version, ensuring token efficiency while maintaining contextual fidelity.
- *Online Developer Portal (DP) Search* Queries search engines with specific issue details, utilizing results from platforms like *StackOverflow* [7] to find optimal solutions. We employ web scraping to create parse trees from HTML documents to extract relevant information that answers the questions queried [14]. For example, when the agent queries '*Spring FileUploadException: Stream ended unexpectedly*', it learns that the corresponding filter needs to be turned off in *StandardMultipartHttpServletRequest.parseRequest*, which it queries using Code Search.
- *Internal Knowledge-base (KB) Search* Uses advanced Retrieval-Augmented Generation (RAG) to locate and return the most relevant documentation and knowledge base entries. We enhance search recall through query expansion and cross-encoder reranking, increasing the likelihood of matching with the most relevant documents [15].

- *Graph Understanding Tool*: Utilizes an internal Neo4j [16] graph where nodes represent microservices, databases, API gateways, and messaging queues, with relationships such as 'connected', 'publishes', or 'consumes'. This tool provides a high-level architectural flow of how the organization's microservices are interconnected to help analyze dependencies and relationships specifically when the root cause arises from components outside the immediate codebase.
- *Git Metadata*: Identifies relevant code changes likely related to the issue and sends Git change metadata when relevant. It ranks code changes through an election process [17]: separate LLM queries are sent, each containing 20 changes, with instructions to select the five most likely error-causing changes. This process is repeated for all outputs combined to return the top five changes.
- *Prompt Engineering Strategy*: DebugMate employs structured prompt templates based on ReAct and Tree-of-Thought methodologies. Prompts follow a "Thought - Action - Action Input" format, which facilitates agent planning, tool invocation, and iterative refinement of reasoning. The prompt system is engineered with safety rules such as requiring multiple branches for ambiguous root causes, rejecting vague inputs for code queries, and enforcing a minimum context-gathering cycle before hypothesis generation. To handle low-quality or underspecified user queries, the prompt explicitly instructs the agent to avoid premature final answers and to prefer using tools like StackOverflow and Confluence search for broader contextualization. The agent is also guided not to proceed with code queries unless definitive function/class names are present, reducing failure from under-specified tool usage.

These tools work together to provide the necessary context for DebugMate to identify and resolve complex issues more accurately. To optimize the process, DebugMate employs the Tree-of-Thoughts architecture, which systematically explores multiple reasoning paths [10]. The ToT framework integrates insights from the primary tools described above to handle complex code execution paths, ensuring that DebugMate can investigate each possible solution thoroughly for evaluating potential problem areas.

2.2 Hypothesis generation and solution formulation

DebugMate utilizes Tree-of-Thoughts to maintain focus while exploring multiple reasoning paths and handling parallel paths of code implementation separately. If there are multiple parallel function calls that need review, the agent branches by calling the Tree-of-Thoughts tool. This approach significantly improves performance by keeping the agent focused on individual branches instead of holding potentially irrelevant information from previous parallel function calls [18].

We answer the following four research questions described in [10] for instantiation of ToT:

- RQ 1. How to decompose the intermediate process into thought steps? We instruct the agent to respond in a structured "Thought, Action, Action Input" format. This method breaks down the process into manageable steps, allowing exploration of various code modules while effectively assessing progress toward solving the problem.
- RQ 2. How to generate potential thoughts from each state? When there are multiple parallel code modules the agent needs to review, it generates instructions and

instantiates a new agent for each module. In the query, it includes possible reasons why the issue could be related to that particular module.

- RQ3. How to heuristically evaluate states? After each branch reaches its conclusion, the parent agent evaluates if it should be added to its list of hypotheses. It uses the confidence score and solution description returned for each conclusion to select the most appropriate reasons with solutions.
- RQ4. What search algorithm to use? DebugMate employs a depth-first search to explore code modules ordered by priority. After each branch concludes, it decides whether to continue its search through other branches or return the final answer reached, depending on the confidence score.

2.3 Retrieval-augmented generation implementation

DebugMate uses a Retrieval-Augmented Generation (RAG) pipeline to incorporate relevant internal documentation into the debugging process. The system is built on LangChain's RetrievalQA framework, with OpenAI's embedding models used to semantically match queries to documents. When queries are underspecified or ambiguous, DebugMate can expand them using few-shot prompting by generating more specific variations that improve retrieval coverage. This is especially helpful when documentation is sparse or terminology varies across teams.

3 Problem formulation

In large production systems, debugging can be a challenging and time-consuming process due to the need to understand the intricate interactions between various components and the vast amount of data involved [19]. Additionally, reproducing and isolating the issue can be time-consuming and challenging. Automated debugging tools have been designed to quickly identify, isolate, and fix issues in complex codebases, saving time and reducing human error. However, large organizations often possess extensive documentation and historical data on previous issues, which automated debugging tools fail to contextualize. DebugMate uses this information to either function as a black box and provide the user with a final list of hypotheses from a single user query, or optionally ask for user input at key turning points of the search execution, such as when creating branches with low confidence scores. The user can also alter the search path taken, by adding branches or terminating them early, depending on additional external context when available.

3.1 Workflow

In managing on-call issues, the workflow begins when the user submits a query explaining the issue along with any additional details available (refer to Fig. 1 for a detailed workflow diagram). This query, along with an optional stack trace, is processed by the Path Tool to look for specific code modules mentioned so that Code Search is equipped with the necessary modules and class hierarchies that the agent needs to query. Additionally, the agent formats the initial prompt, combining instructions for ReAct [9] and Tree-of-Thoughts branching [10], along with descriptions of the tools available, ensuring the agent receives all necessary context to begin the debugging process efficiently. Next, this initial prompt is sent to the LLM, which responds with a single thought, the name of the tool it wants to query, and an input for the tool. Every LLM output is stored

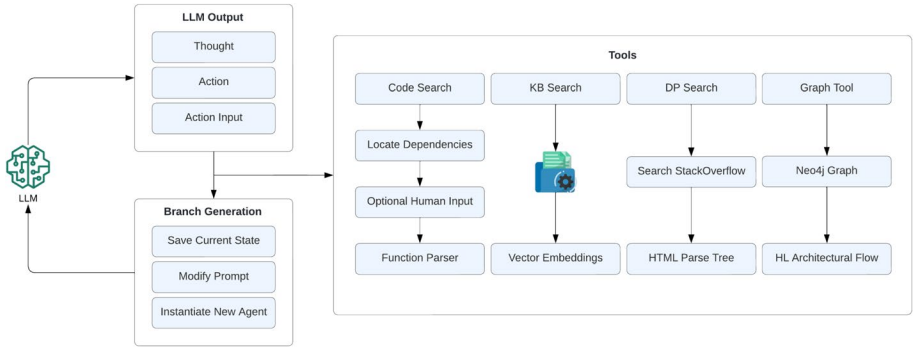


Fig. 1 Workflow diagram

as an intermediate step, which, along with the tool output, is collectively passed back into the LLM as the next input. When the LLM wants to branch, its action is 'ToT Tool', and action input includes all the code modules to search in a JSON string. The ToT tool orders the modules to be searched based on priority, and instantiates a new agent for every branch. It ensures that each branch starts with the same intermediate steps and that the next branch does not include steps from the previous branch that are likely unrelated. It optionally includes the final answer from previous branches when they seem to be important or related, as determined by the agent. In case the current branch concludes with a high confidence score, the agent decides to end the search and return the final response with a list of hypotheses generated so far.

3.2 Debugging unfamiliar external frameworks

One of our primary goals is to specifically assist developers in efficiently debugging unfamiliar code. Modifications in previously stable libraries can cause unforeseen bugs that disrupt an entire software platform. For example, a change was introduced in Spring Framework 6.0 where the trailing slash matching configuration option was deprecated and its default value was set to false [20]. This can cause previously functionally correct URLs to stop working after upgrading to a newer Spring Boot version. Identifying and resolving such issues can be time-consuming, especially when the new version is recently released and changes are not well documented online or within the team's knowledge base.

DebugMate leverages its search capabilities within the codebase and documentation to quickly identify and solve such issues. For example, with a query as simple as '404 error when URL ends with a slash for previously working URLs', DebugMate efficiently searches the organization's Confluence Space [21] and codebase, identifying recent Java and Spring Boot migrations. It then searches official documentation and developer forums to retrieve necessary information and suggests code changes required to either re-enable trailing slashes– calling *configurer.setUseTrailingSlashMatch(true)* in *WebConfiguration.configurePathMatch*– or to adjust the code where URLs are generated.

4 Evaluation

To evaluate DebugMate's performance, we need a benchmark that accurately reflects real-world production issues in large organizations. For this purpose, we use Observe's historical reliability incidents as our benchmark, specifically focusing on P0 and P1 issues with the longest 10% of turnaround times for resolution by developers. Additionally, to

Table 1 Debugging performance on observe AI incidents and bugs from 5 Java-based open-source repositories

	Time (s)	Avg. Depth	KB/DP Used	Err. Identified	Err. Fixed
DebugMate	183.45	6.14	46.15%	83.33%	83.33%
Chain-of-Thoughts	117.12	4.00	28.57%	57.14%	50.00%
Without Stack Trace	151.04	5.14	57.14%	64.29%	57.14%
Without KB/DP Search	172.06	7.56	0	71.43%	64.28%

Table 2 Debugging Accuracy of GPT-4-0613, Human SEs, and DebugMate on ObserveAI's benchmark of historical reliability incidents and DebugBench Logical Errors

Performance	GPT-4-0613	DebugMate
DebugBench	75%	82%
ObserveAI	58%	83%
Open Source	42.8%	71.4%

test on a diverse range of real-world bugs, we analyzed and collected bug reports and issues from five prominent Java-based open-source repositories - Elasticsearch [22], Kestra [23], Dubbo [24], OpenSearch [25], and Jenkins [26] - and documented the results in our GitHub repository [1].

First, we tested DebugMate on logical errors in DebugBench, an LLM Debugging Benchmark [27], and achieved 10% higher accuracy compared to GPT-4-0613. This improvement stems from the agent breaking down the debugging process into logical steps, utilizing ReAct prompts to reason through actions before implementing code changes.

For a comparison of both their capabilities in debugging large production systems, we provided all available resources - stack traces, logs, codebase, and documentation - and tested them on Observe's benchmark. The results are summarized in Table 2.

We assessed the results by classifying them as successful if they correctly identified the module where the root cause is located and specified the exact code changes needed to resolve the issue. We also recorded debugging time, the depth of code modules investigated, and the usage of each tool provided to the agent. Additionally, we conducted experiments involving DebugMate's full capabilities, as well as scenarios where key tools were excluded. The results are summarized in Table 1.

DebugMate identified and fixed about 80% of the errors. It reviewed 6 function calls on average, ranging from 3 to 17. The process took 183 s on average. GPT-4-0613's accuracy on the same benchmark was about 60%, and debugging with it took 30% longer than with DebugMate.

When the Tree-of-Thoughts functionality was removed from DebugMate, and Chain-of-Thoughts was implemented instead, the errors identified were reduced to 57% and errors fixed to 50%. This is because context from previously unrelated branches needed to be held in memory, which caused the agent to lose its focus on the current path being searched [28]. This loss of focus led to reviewing fewer function calls on average– 3.4 down from 6 with Tree-of-Thoughts– affecting its ability to reach the depth where the error actually occurred.

4.1 Analysis of real-world debugging performance

To assess DebugMate's generalizability beyond internal datasets, we evaluated it on real-world issues from widely used open-source Java repositories including Dubbo, Kestra,

Elasticsearch, OpenSearch, and Jenkins. These issues span a variety of types of bugs, such as type mismatches, configuration errors, data handling bugs, and concurrency issues.

DebugMate successfully identified the root cause in 14 of 16 cases, and proposed correct or partially correct fixes in 11 of those. For example, in Dubbo issue 13689, it correctly traced an NPE to improper return typing and proposed the same fix as the project's pull request [29]. In Kestra issue 5535, it pinpointed a type compatibility issue and suggested using Float.valueOf(...), aligning with the fix merged upstream [30]. In Elasticsearch pull 115474, DebugMate recommended checking null conditions before function invocation, matching the final patch [31].

4.2 Limitations

Several cases revealed the system's current limitations. In OpenSearch issue 5035, DebugMate failed to locate the right module due to shallow context injection and spent multiple iterations exploring unrelated files [32]. In Dubbo issue 14740, although the agent correctly identified the error as a failed extension instantiation, it failed to narrow down into the specific class-level cause [33]. Another limitation emerged in Jenkins, where the agent could not resolve a race condition caused by concurrent UI rendering and backend state updates, as the required observability signals were unavailable in static context. These results highlight that while DebugMate is effective in many realistic debugging tasks, its performance still depends on the clarity of stack traces, depth of accessible documentation, and availability of complete code paths.

5 Conclusion

Our research indicates that access to key internal documentation, external knowledge sources, and entire codebases, when provided to an agent employing Tree-of-Thoughts, significantly increases the accuracy of identifying and resolving issues in large production systems. While the current system achieves strong performance across benchmarks, it still requires a human in the loop in real-world scenarios. Software engineers typically initiate the debugging process by providing a natural language query or stack trace, and may occasionally need to guide the agent by clarifying ambiguous inputs or supplying missing code fragments. Additionally, the prompt engineering and retrieval strategies used were tailored to the specific categories of bugs encountered during our experiments, and may require adaptation to handle broader or more novel failure modes.

5.1 Future work

Future development could involve adding code execution capabilities and access to API endpoints to improve system understanding and increase the accuracy of resolving problems. Further, since DebugMate currently supports only Java code, we plan to expand to other languages in the future to address this limitation. While DebugMate is effective at navigating code and documentation, it currently lacks a deeper systems-level understanding and reasoning ability derived from core computer science principles such as memory models, concurrency, or distributed system behavior. Implementing functionality to modify code and test fixes in real-time, as seen in other debugging tools [3], could also be explored. While such tools have shown promise, they often do not utilize

the same approach as DebugMate, presenting a unique opportunity to combine these methodologies for improved debugging outcomes.

Author contributions
R.M., N.R., and S.S.K. contributed to all aspects of this work, including the design and development of DebugMate, conducting experiments, analyzing results, and writing the manuscript. All authors reviewed the final version of the manuscript.

Funding
No funding was received to assist with the preparation of this manuscript.

Data availibility
All open-source experiments conducted during this study are available to ensure transparency and reproducibility. The full set of results and code can be accessed on our GitHub repository: https://github.com/Observeai-Research/DebugMate.

Declarations

Ethics approval and consent to participate
Not applicable

Consent for publication
Not applicable

Competing interests
The authors declare no Competing interests.

Received: 4 February 2025 / Accepted: 30 July 2025
Published online: 15 August 2025

References

1. Observe.AI. Debugmate. 2024. (Accessed 2024) https://github.com/Observeai-Research/DebugMate
2. Karimi M, Barfroush AA. Proposing a dynamic executive microservices architecture model for ai systems. 2023.
3. Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, Press O. Swe-agent: Agent-computer interfaces enable automated software engineering. Adv Neural Inf Process Syst. 2024;37:50528–652 arXiv:2405.15793.
4. 00-Python. Project aware coding assistant using the openai/google api. Published electronically at. 2024. (Accessed: 2024) https://github.com/00-Python/AI-Debug
5. Levin KH, van Kempen N, Berger ED, Freund SN. Chatdbg: An ai-powered debugging assistant. 2024. arXiv: 2403.16354.
6. On-call problems. Published electronically at. 2023. (Accessed: 2024) https://news.ycombinator.com/item?id=36426851
7. Stackoverflow. StackOverflow, 2024. Accessed: 2024 https://stackoverflow.com/
8. Github documentation. GitHub. 2024. Accessed: 2024 https://docs.github.com/
9. Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y. React: Synergizing reasoning and acting in language models. InInternational Conference on Learning Representations (ICLR) 2023. arXiv: 2210.03629.
10. Yao S, Yu D, Zhao J, Shafran I, Griffiths T, Cao Y, Narasimhan K. Tree of thoughts: deliberate problem solving with large language models. Adv Neural Inf Process Syst. 2023;15(36):11809–22 arXiv: 2305.10601.
11. Liu W, Yu A, Zan D, Shen B, Zhang W, Zhao H, Jin Z, Wang Q. Graphcoder: Enhancing repository-level code completion via code context graph-based retrieval and language model. 2024. arXiv: 2406.07003.
12. Guo J, Li N, Qi J, Yang H, Li R, Feng Y, Zhang S, Xu M. Empowering working memory for large language model agents. 2024. arXiv: 2312.17259.
13. Layman L, Diep M, Nagappan M, Singer J, Deline R, Venolia G. Debugging revisited: Toward understanding the debugging needs of contemporary software developers. *2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement* 2013;383–392.
14. Lotfi C, Srinivasan S, Ertz M, Latrous I. Web Scraping Techniques and Applications: A Literature Review, (ResearchGate), 2021;381–394
15. Li M, Zhuang H, Hui K, Qin Z, Lin J, Jagerman R, Wang X, Bendersky M. Can query expansion improve generalization of strong cross-encoder rankers? Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval 2024. https://doi.org/10.1145/3626772.3657979.
16. Neo4j documentation. Neo4j. 2024. Accessed: 2024 https://neo4j.com/docs/
17. Hsu D, Neu M, Farrag M, Kindi, R. Leveraging ai for efficient incident response. 2024. Accessed: 2024 https://engineering.fb.com/2024/06/24/data-infrastructure/leveraging-ai-for-efficient-incident-response/
18. Wolfe CR. Tree of thoughts prompting. Published electronically at. 2024. https://cameronrwolfe.substack.com/p/tree-of-thoughts-prompting
19. Fitzgerald S, Lewandowski G, McCauley R, Murphy L, Simon B, Thomas L, Zander C. Debugging: finding, fixing and flailing, a multi-institutional study of novice debuggers. Comput Sci Educ. 2008;18(2):93–116.
20. Projects, S. Spring boot 3.0 migration guide. Published electronically at. 2024. Accessed: 2024 https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0-Migration-Guide
21. Neo4j documentation. Confluence. 2024. https://support.atlassian.com/confluence-cloud/docs/administer-confluence-cloud-sites/.
22. Banon S, et al. Elasticsearch. 2024. Accessed: 2024 https://www.elastic.co/elasticsearch
23. Kestra A. Apache kestra. 2024. Accessed: 2024 https://kestra.io

24. Dubbo A. Apache dubbo. 2024. Accessed: 2024 https://dubbo.apache.org
25. OpenSearch. Opensearch. 2024. https://opensearch.org
26. Jenkins. Jenkins. 2024. https://www.jenkins.io
27. Tian R, Ye Y, Qin Y, Cong X, Lin Y, Pan Y, Wu Y, Hui H, Liu W, Liu Z, Sun M. Debugbench: Evaluating debugging capability of large language models. 2024. arXiv: 2401.04621.
28. Besta M, et al. Demystifying chains, trees, and graphs of thoughts. 2024. arXiv: 2401.14295.
29. Observe.AI. Debugmate output for dubbo issue 13689. 2024. Accessed: 2024 https://github.com/Observeai-Research/DebugMate/blob/main/open_source_experiments/npe_com_qiniu_http_response.txt
30. Observe.AI. Debugmate output for kestra issue 5535. 2024. Accessed: 2024 https://github.com/Observeai-Research/DebugMate/blob/main/open_source_experiments/float_type_input_subflow_error.txt
31. Observe.AI. Debugmate output for elasticsearch pull 115474. 2024. Accessed: 2024 https://github.com/Observeai-Research/DebugMate/blob/main/open_source_experiments/npe_deployment_stats_8.txt
32. Observe.AI. Debugmate output for opensearch issue 5035. 2024. Accessed: 2024 https://github.com/Observeai-Research/DebugMate/blob/main/open_source_experiments/search_as_you_type_error
33. Observe.AI. Debugmate output for dubbo issue 14740. 2024. Accessed: 2024 https://github.com/Observeai-Research/DebugMate/blob/main/open_source_experiments/requestmappingresolver_instantiation_exception.txt

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.